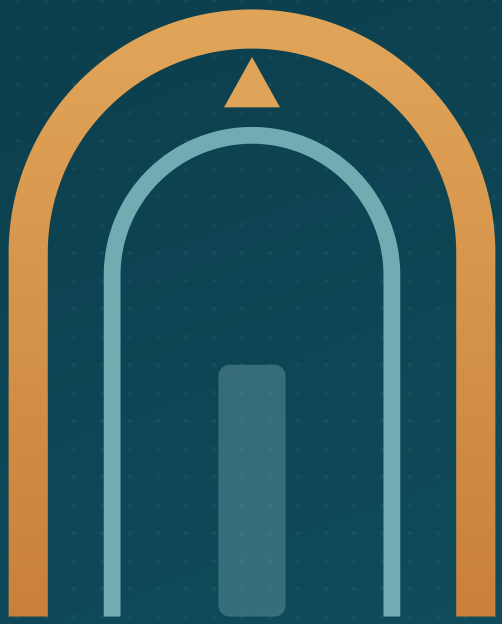


VISUAL DESIGN OVERVIEW

Bawaba

بوابة

A secure, high-performance gateway in front of the Shamel Pro accounting API: unlimited scoped tokens, fine-grained permissions, a firewall that keeps Shamel invisible, and an API that AI agents can build on safely.

Version
0.1 — draft for review

Date
3 November 2026

Companion
Full design reference (PDF)

01 AT A GLANCE

One secure door in front of Shamel

Today every integration logs into Shamel with a real Shamel username and password and can see whatever that user sees. Bawaba becomes the **only** way in: each tool gets its own token with exactly the access it needs, and Shamel becomes unreachable to anyone but Bawaba.

> **"Make as many API tokens as I want, more security and permissions, maybe expanding the API itself, better documentation, and a firewall."**

∞
API tokens, each with its own permissions, IP allowlist and usage stats

0
open ports needed at the Shamel site when the Connector is used

100%
of requests and responses recorded in an encrypted audit log

29/30
Shamel endpoints (all but login) re-exposed as one clean, documented `/v1` API

Origin-hiding firewall
Shamel's IP, host, version and raw errors never leave Bawaba. Cloudflare in front, an outbound-only Connector behind.

Tokens & permissions
Scopes per resource, per voucher type, data scoping (stores, salesmen, customers…) and field hiding. Read-only and write-only presets.

Safe for AI builders
AI agents only ever talk to Bawaba. They can *request* access; nothing goes live until you approve it on the server. Sandbox with fake data.

A better API
Clean `/v1` REST with real status codes, paging, search, combined endpoints, webhooks and idempotent writes — plus a legacy-compatible mode.

Built for load
Go, stateless nodes, streaming JSON, request coalescing. Always-live data; the hardware should be the only bottleneck.

Full audit, optimized
Every call recorded in full, encrypted — with de-duplication and compression so storage stays small.

Inside this overview

01	At a glance	2	02	Why a gateway	3	03	Architecture	4
04	Security layers	5	05	Shamel stays invisible	6	06	The Connector	7
07	Tokens & permissions	8	08	AI agents	9	09	Safe writes	10
10	The API	11	11	Endpoint map	12	12	Performance	13
13	Audit log	14	14	Webhooks & guidance	15	15	Dashboard & CLI	16
16	Deployment	17	17	Delivery plan	18	18	Questions for ISCO	19
19	Decisions	20						

02 WHY A GATEWAY

Shamel's API today vs. with Bawaba

Shamel Pro's Web API works, but it was built for one trusted integration, not for many tools with different needs. Bawaba keeps Shamel exactly as it is and fixes everything around it.

Topic	Shamel API today	With Bawaba
Credentials	One real Shamel username + password per integration; token from `/Main/Login`	Unlimited Bawaba tokens; tools never see Shamel credentials. In Connector mode only the Connector can even decrypt them.
Permissions	Only Shamel's own salesman scoping; `getAll=true` lifts it for the customer list	Scopes, voucher-type rights, data scoping and field hiding per token; deny by default
Exposure	Shamel server must be reachable by every integration	Shamel reachable only by Bawaba — through an outbound Connector tunnel (no open ports) or an IP-locked direct URL
Errors	Every error is HTTP 500 with raw details (can leak internals)	Proper status codes and stable error codes in English + Arabic; internals only in the encrypted audit log
Parameters	Filters passed as HTTP headers; writes as a JSON string inside a form field (`obj=`)	Query parameters and JSON bodies, validated against a strict schema
Large lists	Everything returned at once, no paging	Cursor paging over a 60-second snapshot, search (Arabic-aware), sorting
Naming	Typos and inconsistencies (`sale_parice1` , `orgnization_code` , `refrenceId`)	Consistent snake_case with typos fixed, documented in OpenAPI 3.1
Duplicates	A retried request can post the same invoice twice	`Idempotency-Key` required on every `/v1` write; optional human approval queue
Change tracking	Clients must poll and diff themselves	Signed webhooks for customers, items, stock, vouchers, order status (M3; Bawaba checks Shamel every 5 min)
Visibility	No per-integration usage stats or API audit trail	Per-token analytics, full encrypted audit, e-mail alerts
Documentation	A 20-page PDF	Interactive bilingual docs, error catalogue, `llms.txt` for AI agents, sandbox

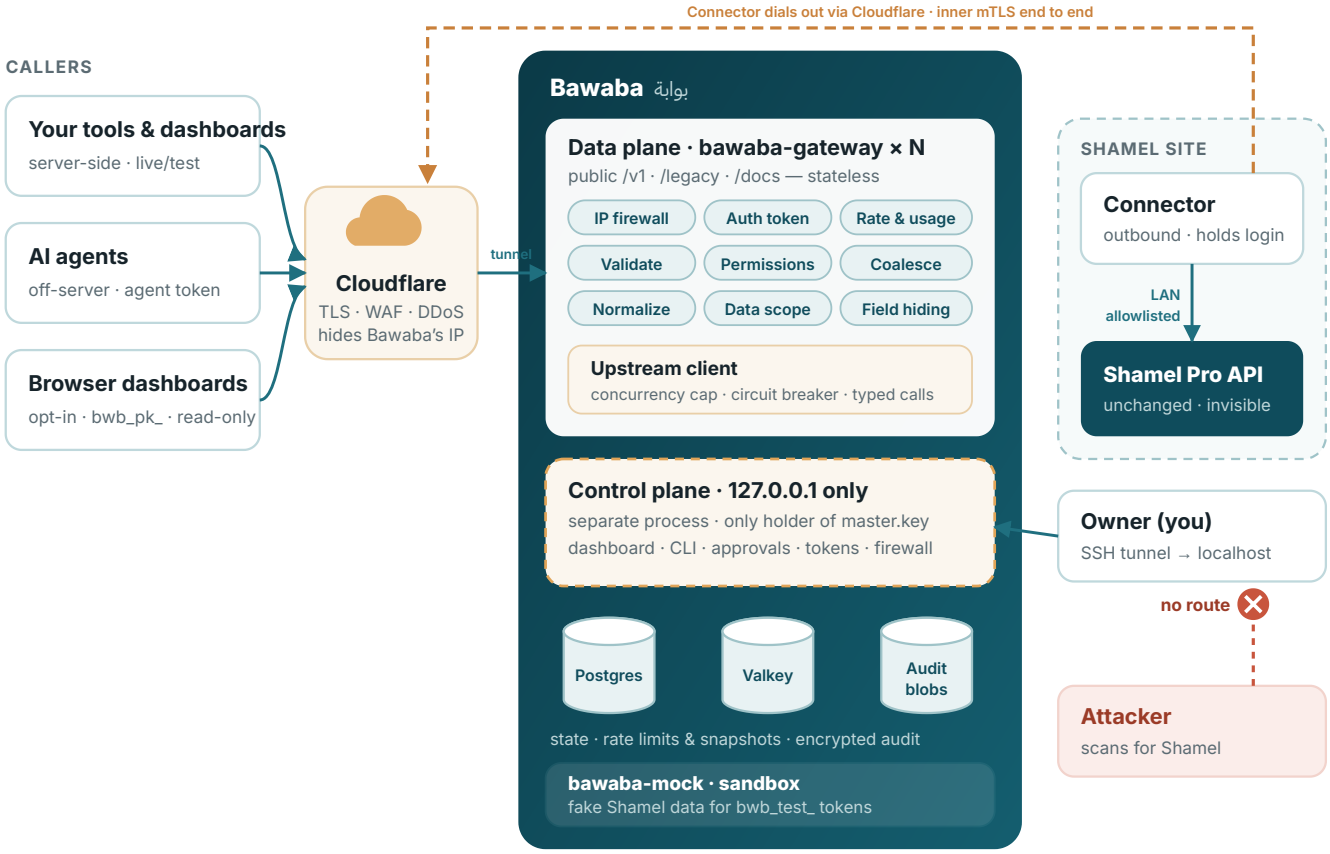


Nothing changes inside Shamel. Bawaba is a separate layer: Shamel keeps running exactly as today, and existing integrations can migrate gradually through Bawaba's legacy-compatible paths (same requests at a new URL, with a Bawaba token instead of the Shamel login).

03 ARCHITECTURE

How the pieces fit together

Every request enters through Cloudflare and passes Bawaba's pipeline. It reaches Shamel through the Connector's outbound tunnel, or an IP-locked direct link where no Connector can be installed. The admin side runs as a separate process bound to the server itself, with no public route.



request path · Connector tunnel (initiated from the Shamel site) · blocked

Data plane

Stateless Go nodes behind Cloudflare. Add nodes to scale; any node can serve any request.

Control plane

A separate process bound to `127.0.0.1`, the only holder of `master.key`; you reach it through an SSH tunnel. Approvals and live-token creation happen only here.

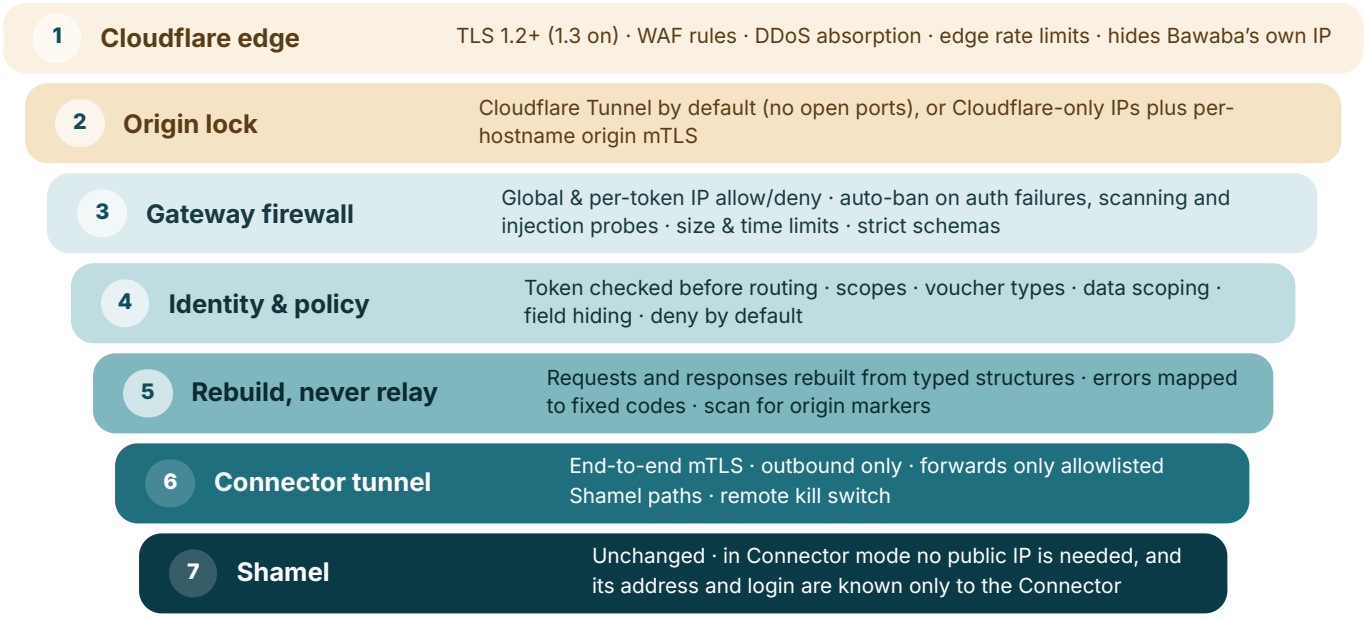
Connector

Small service next to Shamel that dials *out* to Bawaba and holds the Shamel login. Shamel needs no public IP and no open port.

04 SECURITY & FIREWALL

Layers between an attacker and Shamel

Cloudflare, the gateway and the Connector each enforce their own allowlist, so a failure in one is contained by the next. The outer layers hide where things are; the inner ones decide what each token may do. In Connector mode, Shamel is never reachable from the internet at all.



Accepted trade-off: Cloudflare terminates public TLS, so it can see regular API traffic. It never sees Shamel credentials or the contents of the Connector tunnel.

What an attacker can see

- A Cloudflare IP address, not Bawaba's and never Shamel's
- The same `401` on every API path (`/v1` , `/legacy`), real or not
- No `Server` , `X-Powered-By` or version headers
- Generic error codes, never upstream text, paths or SQL

What a stolen token can do

- Only what its scopes, data scope and hidden fields allow
- Only from its IP allowlist, if one is set
- Writes can be held in the approval queue for your review
- Revoke it instantly; every node drops it within seconds

Encryption & keys

Two key files
master.key (control plane, backed up offline) + data.key (data plane, re-issuable); root-only, never in the DB

→

Key-encryption keys
one per plane (control, data), wrapped by the key files; rotatable

→

Data keys · AES-256-GCM
one per purpose: snapshots · pending secrets · Shamel passwords (direct mode). Audit uses hourly keys only the control plane can open

API tokens

256-bit random with a `bwb_` prefix and checksum, so secret scanners can spot leaks without false alarms. Stored only as an HMAC hash and shown once.

Stolen backup?

Without the two key files (never put in backups), a database or blob-store copy can't be read. Connector-mode Shamel passwords can't be read even with them.

Owner login

Password hashed with argon2id plus optional TOTP. The dashboard is reachable only on `127.0.0.1` through SSH.

05 SECURITY & FIREWALL

Shamel stays invisible

Eight principles govern the code. The core one: Bawaba **rebuilds** every request and response from its own types and never copies upstream bytes across, so Shamel's internals can't be copied out, and a marker scan flags any stray ones.

P1
Origin can't be addressed
No byte a client sees comes from Shamel's address, software, headers or errors.

P2
Rebuild, never relay
Typed structs in both directions, including the legacy mode.

P3
Deny by default
Closed allowlists for routes, fields, headers, origins and webhook targets.

P4
Authenticate first
Anonymous callers get the same 401 on every API path and never trigger a Shamel call.

P5
Fail closed
Missing key, unknown peer or stale policy means the request is rejected, never passed through.

P6
Owner power stays on the server
Approvals, firewall changes, live-token creation and key operations exist only in the control plane.

P7
Bounded blast radius
Each component's compromise is documented and contained.

P8
Encrypted everywhere
TLS/mTLS on every network hop (Shamel's LAN hop if it has HTTPS); AES-256-GCM at rest; keys apart from data.

The same failure, before and after

SHAMEL DIRECTLY · LEAKS INTERNALS

```
HTTP/1.1 500 Internal Server Error
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET

System.Data.SqlClient.SqlException: Invalid
object name 'dbo.tblCustomers'. at
C:\inetpub\Shamel\Api\… on SHAMEL-SRV
```

Illustrative example of the kind of response the manual describes (every error is HTTP 500 with details).

THROUGH BAWABA · NOTHING TO LEARN

```
HTTP/1.1 502 Bad Gateway
Bawaba-Request-Id: req_01JC2S4B…

{"error": {
  "code": "upstream_error",
  "message": "The accounting system could not complete
the request.",
  "message_ar": "تعذّر على النظام المحاسبي…",
  "request_id": "req_01JC2S4B…",
  "docs_url": "…" }}
```

The details (with Shamel's address scrubbed) go to the encrypted audit log, readable only on the server.

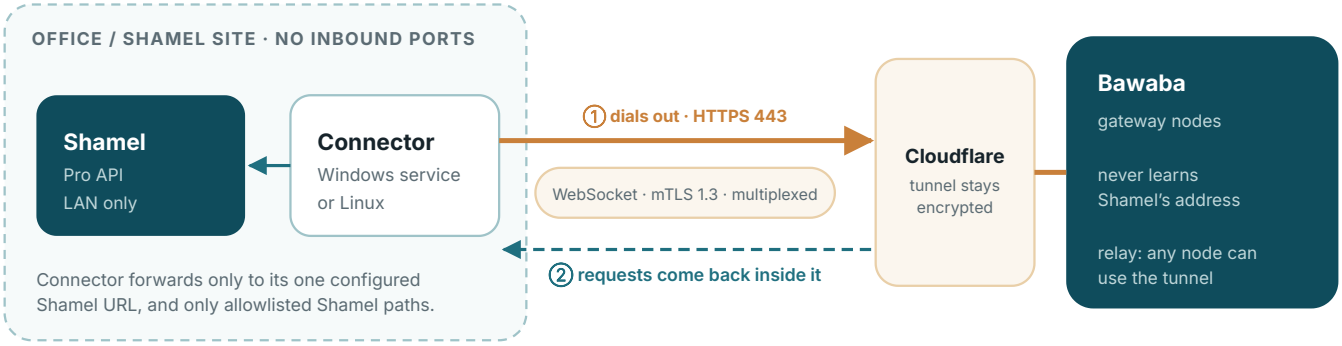
28 **leak vectors listed, closed, and checked by an origin-leak test suite that blocks every pull request**

error bodies · business error text · Bawaba's own errors · SQL errors · server headers · NTLM challenges · paths in payloads · redirects · DNS records · TLS certs / CT logs · timing · Connector visibility · docs & OpenAPI · tool-visible logs · webhooks · CORS · health & metrics · version strings · reflected input · internal ids · cursors · load signals · sandbox cross-talk · trace ids · image EXIF · e-mail headers · legacy passthrough · Shamel credentials

06 REACHING SHAMEL

The Connector: no inbound ports needed at the Shamel site

A small Bawaba service runs next to Shamel, on Windows or Linux. It makes an **outbound** HTTPS connection to Bawaba, and Bawaba sends its Shamel requests back through that connection. Nothing on the internet can start a connection to Shamel or the Connector.



Enrollment

A one-time enrollment code gives the Connector its own mTLS certificate from Bawaba's internal CA. You confirm it on the server before it goes live.

Holds the Shamel login

The Shamel password is sealed so only the Connector can open it, and the Connector logs in to Shamel itself. Even a compromised gateway can't read the password and can only request allowlisted paths.

Kill switch

`bawaba connector kill` cuts it off within a second, for good (restarting needs a new enrollment). The same certificate from two IPs, or a network change, raises a critical alert and suspends it.

Two ways to reach Shamel

Mode	How it works	When to use
Connector PREFERRED	Outbound tunnel from the Shamel site; Shamel stays on the LAN with no public IP	You can install a small service next to Shamel
Direct URL	Bawaba calls Shamel directly, encrypted via site-to-site WireGuard or pinned HTTPS (plain HTTP over the internet is refused); Shamel's firewall allows only Bawaba	Installing the Connector isn't possible (e.g. Shamel hosted off-site)

Writes through the Connector are off until you enable them locally on the Shamel host, a backstop even if the gateway were compromised. Replacing a Connector means re-entering the Shamel passwords, since the gateway can't decrypt them; use `suspend` for temporary stops.

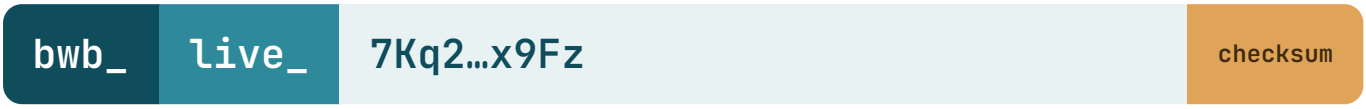
⇄ **High availability.** Run two Connectors. By default one serves each Shamel login and the other takes over if it drops; any gateway node reaches it through the node holding its tunnel.

! **If the tunnel drops.** Tools get a clean `503 upstream_unavailable` with `Retry-After`; the Connector reconnects with backoff, and you get an e-mail if it stays down for 2 minutes.

07 TOKENS & PERMISSIONS

Every tool gets exactly the access it needs

Create as many tokens as you like. Each one carries its own permission profile, checked on every request and denied by default. Revoking a token takes effect on every node within seconds.



Prefix marks it as a Bawaba token, so leak scanners (e.g. GitHub) can recognise it

Environment `live_` = real Shamel · `test_` = sandbox; browser tokens add `pk_` (`bwb_pk_live_…`)

256-bit secret shown once, stored only as an HMAC hash

Checksum lets scanners and Bawaba reject typos and look-alikes instantly

Four permission layers, applied in order

1 **Scopes** · which resources, read or write

`customers:read` `items:read` `stock:read` `vouchers:read` `reports:read` `webhooks:write`

2 **Voucher types** · which documents it may create

`order ✓` `receipt ✓` `sales_invoice ×` `journal ×`

3 **Data scope** · which rows it can see

`stores 1, 2` `salesman S07` `currency NIS` `price classes` `customers` `voucher types`

4 **Field hiding** · which fields are stripped from responses

`items.purchase_price` `customers.balance` `customers.credit_limit`

Plus, per token

- IP allowlist
- Legacy-mode on/off
- Writes need your approval
- Optional expiry (none by default)
- Own rate limits and usage stats

Presets

Templates that expand into explicit grants:

`read-only` `write-only` `full access` `sales dashboard` `stock monitor` `accountant reports`

Shamel login per token

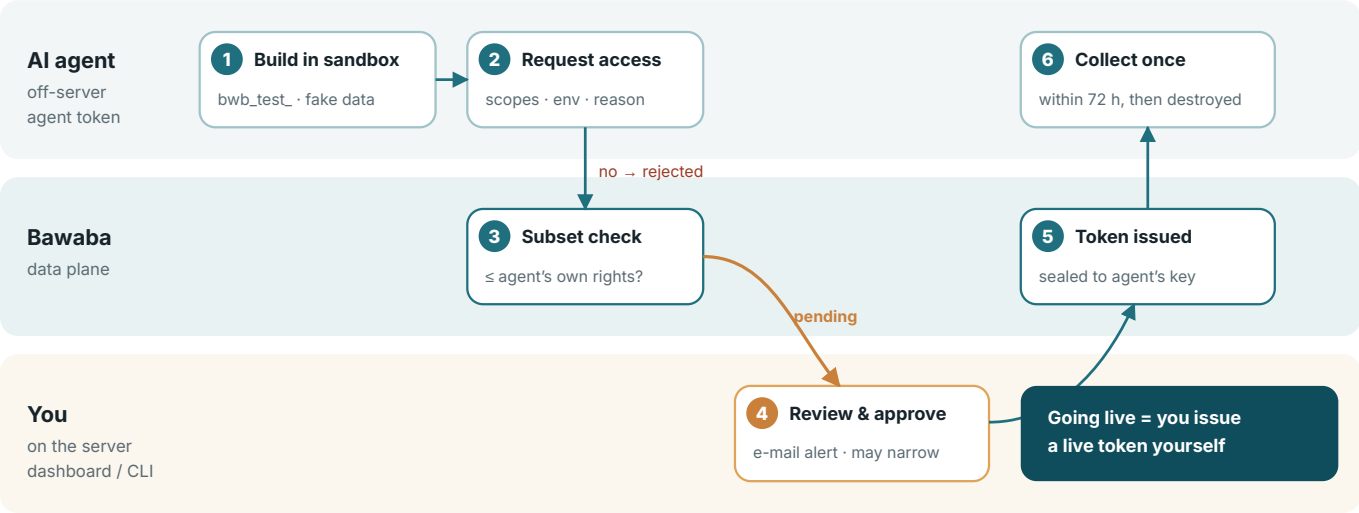
Tokens use the default Shamel service account, or can be pinned to a specific Shamel user, such as a salesman, to keep Shamel's own salesman filtering. Shamel's "all customers" override is never used without explicit permission.

i **Nothing to probe.** Rows outside a token's data scope are simply absent, and counts include only visible rows. An out-of-scope customer returns the same `404` as a customer that doesn't exist. Write permission never implies read permission, which is what makes "write-only" tokens possible.

08 AI AGENTS

AI can ask. Only you can say yes.

AI agents build your tools, but they never touch Shamel directly and never see its credentials or address. An agent works against a sandbox with fake data by default and can **request** tokens. Nothing goes live until you approve it, on the server.



Rules an agent can't get around

- It can never request more than its own token holds: scopes, data scope, hidden fields, IP rules, environment and expiry are all checked
- It can't approve anything; approval exists only on the server
- The check runs again at approval time, against the agent's rights at that moment
- At most 20 pending and 50 requests a day per agent, and widening requests can't be bulk-approved, so you won't be worn down into clicking "yes"

Safe defaults

- Agent tokens start in the **sandbox** (`bwb_test_` , built-in mock Shamel with Arabic fake data)
- Tokens born from agent requests always start with writes held for your approval; only you can turn that off
- You can narrow a request before approving, never widen it
- Collecting a token from a different IP than the request came from raises an alert

The sandbox

```
GET /v1/customers?limit=2
Authorization: Bearer bwb_test_…

200 OK
Bawaba-Environment: sandbox
{"data":[{"code":"C1001",
  "name":"شركة النور للتجارة",
  "city_name":"نابلس", …}],
 "page":{"next_cursor":"…"}}
```

Same contract as live

Identical endpoints, fields, errors and limits, so a tool built in the sandbox works unchanged with a live token.

Realistic fake data

A deterministic dataset with Arabic names, multiple currencies, stores, salesmen and vouchers.

Can't cross over

A test token can never reach live Shamel: the token prefix and its stored environment must agree, or the request fails.

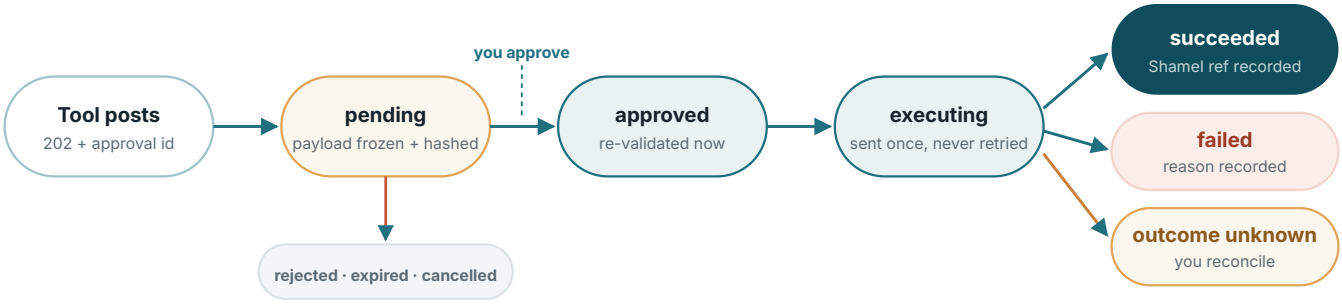
Reset anytime

Each test token gets its own sandbox space (writes kept 7 days) that can be reset; sandbox audit bodies are kept 14 days.

09 SAFE WRITES

A retried invoice never posts twice, and can wait for you

Writes (new customers, items, cost centers and all 13 voucher types) come in milestone 2. Every `/v1` write must carry an `Idempotency-Key`, so a retry with the same key never creates a duplicate. For tokens you choose, writes wait in an approval queue (live items expire after 7 days) until you have looked at them.



You see a real document

The dashboard shows the voucher as a business document (type, customer, lines, totals, currency) next to the raw JSON.

Never double-posted

Same key + same body returns the original result. Same key + different body returns `409`. A write whose outcome is unknown raises a critical alert instead of a retry.

Undo-friendly audit

Every write keeps its full payload, who sent it, who approved it, and Shamel's reference id, so you can find it and correct it in Shamel (e.g. with a return or credit note).

What a tool sees

```
# tool
POST /v1/vouchers/order
Authorization: Bearer bwb_live_…
Idempotency-Key: 4f1c-…-order-1093

{"customer_code":"C1001","currency_code":"NIS","rat
e":"1",
 "lines":[{"item_code":"A-100","quantity":"12",
           "price":"8.5","store_id":1}]}
```

```
# Bawaba
HTTP/1.1 202 Accepted
Location: /v1/approvals/apr_51

{"data":{"approval":
{"id":"apr_51","status":"pending",
  "expires_at":"2026-11-10T12:00:00+02:00"}}}

# later: GET /v1/approvals/apr_51 → "succeeded",
# "reference_id":"O-10931"  (or a webhook event)
```

Batches

Up to 100 writes per request, run one at a time in order and not all-or-nothing (Shamel has no transactions); each has its own key; stops at the first error unless you turn that off.

Retry rules

A key is kept for 24 hours after the final result. Validation failures release the key, so the same key can be reused once the body is corrected.

Limits & fail-safes

Per tool: 50 pending and 200 new approvals a day on live. If the idempotency store is down, nothing is sent: `503 idempotency_unavailable`.

10 THE API

A clean /v1 API, with a bridge for old integrations

Query parameters, JSON bodies, real status codes. A per-token **legacy mode** keeps Shamel's paths, so old integrations just switch URL and use a Bawaba token instead of the Shamel login. Both share the same permissions, firewall and audit.

SHAMEL TODAY

```
GET /Customer/GetCustomers
username: api-user
token: 9f8e…   # Shamel session
code: C1001
getBalance: true
getContacts: true

→ 200 [ { "code":"C1001", "price_class": 2.0,
        "notPayedCheqsBalance": … } ]
→ any error: 500 + raw exception text
```

BAWABA /V1

```
GET /v1/customers/C1001?include=balance,contacts
Authorization: Bearer bwb_live_…

→ 200 {"data":{ "code":"C1001",
        "price_class_id": 2,
        "credit_limit": "5000", … }}
→ 404 not_found · 403 insufficient_scope
→ 503 upstream_unavailable + Retry-After
```

Consistent naming

snake_case everywhere and Shamel's typos fixed: `sale_parice` → `sale_price`, `orgnization` → `organization`, `refrenceId` → `reference_id`.

Exact money

Amounts, prices, quantities and rates are decimal **strings** (`"8.5"`), so no JavaScript tool can lose precision through floating point.

Dates & text

`YYYY-MM-DD` dates, RFC 3339 timestamps, Asia/Hebron business time, full UTF-8 Arabic with Arabic-aware search.

Bilingual errors

Stable `code` + English `message` + Arabic `message_ar` + `request_id` + a docs link for every error.

Lists that page

`?limit=&cursor=` over a 60-second snapshot, plus filters, sorting, `q=` search and `updated_since`.

Combined endpoints

Customer overview (profile, balances, recent vouchers, statement summary), stock across all stores, item overview.

Status codes at a glance

Status	Typical codes	Meaning for the tool
400	`invalid_request` `unknown_parameter`	Fix the request; unknown fields are rejected, not ignored
401	`unauthenticated` `invalid_token` `token_expired`	Same answer on every path, so routes can't be discovered
403	`insufficient_scope` `ip_blocked` `origin_not_allowed`	Not allowed (scope, IP or origin); missing scopes are listed in the details
404	`not_found`	Missing *or* outside the token's data scope (indistinguishable)
409	`snapshot_expired` `cursor_invalidated`	The 60-second list snapshot ended or permissions changed: start again from page 1
409/422	`idempotency_conflict` `validation_failed` `upstream_rejected`	Business or consistency problem, explained in English and Arabic
429	`rate_limited` + `Retry-After`	Slow down; warnings usually arrive well before this
502–504	`upstream_error` `upstream_unavailable` `upstream_busy` `upstream_timeout`	Shamel side; retry reads (on 503 after `Retry-After`); retry writes only with the same `Idempotency-Key`

11 THE API

Endpoint map

Every Shamel endpoint except login has a `/v1` home. Milestone 1 delivers all the reads; writes, agent access requests and webhooks follow.

Master data — `master_data:read`

GET	/v1/currencies
GET	/v1/currency-rates
GET	/v1/banks
GET	/v1/bank-branches
GET	/v1/cities
GET	/v1/stores
GET	/v1/price-classes
GET	/v1/salesmen

Customers — `customers:read`

GET	/v1/customers
GET	/v1/customers/{code}
GET	/v1/customer-balances
GET	/v1/customers/{code}/balances
GET	/v1/customer-prices
GET	/v1/customers/{code}/prices
GET	/v1/customers/{code}/overview ★

Items — `items:read`

GET	/v1/items
GET	/v1/items/{code}
GET	/v1/items/{code}/overview ★
GET	/v1/item-groups · /tree
GET	/v1/item-serials
GET	/v1/items/{code}/serials
GET	/v1/item-images
GET	/v1/items/{code}/images
GET	/v1/media/{media_id}

Stock — `stock:read`

GET	/v1/stock
GET	/v1/items/{code}/stock ★

Vouchers (read) — `vouchers:read`

GET	/v1/vouchers
GET	/v1/vouchers/{code}
GET	/v1/voucher-summaries
GET	/v1/deliveries/prepared
GET	/v1/order-status

Reports — `reports:read`

GET	/v1/reports/account-statement
GET	/v1/reports/last-prices
GET	/v1/reports/cost-center-statement

cost-center report also needs `cost_centers:read`

Writes · M2 — `…:write`

POST	/v1/customers
POST	/v1/items
POST	/v1/cost-centers
POST	/v1/vouchers/{voucher_type}
POST	/v1/batch
GET	/v1/approvals/{id}

Self-service · agents (M2) — `any token`

GET	/v1/me · /v1/me/requests
GET	/v1/event-types
POST	/v1/access-requests · M2
GET	/v1/access-requests/{id} · M2
POST	…/{id}/claim-secret · M2

Webhooks · M3 — `webhooks:*`

CRUD	/v1/webhooks
POST	…/{id}/test
POST	…/{id}/rotate-secret
GET	…/{id}/deliveries
GET	…/{id}/events (pull)

13 voucher types

Each create permission is separate: `vouchers:write:<type>`

`order · طلبية` `sales_invoice · فاتورة مبيعات` `sales_return · مردودات مبيعات` `receipt · قبض` `payment · صرف` `journal · قيد` `credit_note` `debit_note` `sales_delivery` `purchase_delivery` `internal_delivery` `stockout` `returned_pending_sales_delivery`

Legacy mode · `/legacy/<Controller>/<Action>`

For existing integrations: the same Shamel paths and shapes, authenticated with a Bawaba token (never Shamel credentials). Off by default and allowlisted per token. Responses are still rebuilt, filtered and audited; nothing is passed through raw. Its reference is served only to entitled tokens at `GET /v1/docs/legacy`; legacy writes arrive in M2.

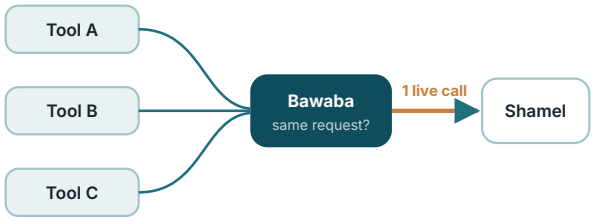
★ marks combined endpoints that gather several Shamel calls into one response.

12 PERFORMANCE

Always live, and still fast

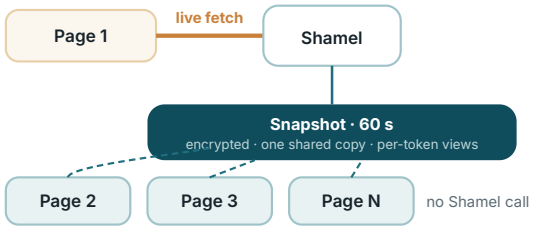
You chose **always-live** data, so Bawaba never serves stale figures. It saves work in only two safe ways: identical simultaneous requests share one live call, and a 60-second snapshot keeps the pages of one list consistent.

Request coalescing



Merged only when the Shamel login and the Shamel call are identical; each token's own permissions are applied afterwards, so sharing never widens what it sees.

Pagination snapshots



Shamel returns whole lists at once; Bawaba pages, filters, sorts and searches that one fetch, with Arabic-aware search.

Targets

≤5 ms
p99 overhead for small responses, excluding Shamel and queue time

5k+
small reads/s per 4-vCPU node, full audit on (Shamel time excluded)

10k+
cursor-page reads/s per node

≥85%
of linear scaling up to 8 nodes

Why hardware stays the only limit

- Stateless Go nodes: add nodes to scale, no sticky sessions
- Streaming JSON: huge lists are processed record by record, never held as one big object; memory stays a fraction of the list size and is capped per node
- Permissions compiled in memory and audit written in background batches: no database round-trip on reads

Protecting Shamel itself

- A global cap on simultaneous Shamel calls: requests queue instead of overloading the server
- Circuit breaker when Shamel is down or slow
- Retries only on safe reads; one shared re-login when Shamel's session expires

When something goes wrong

Failure	What Bawaba does	Tools see	E-mail
Shamel down	Circuit breaker opens; webhook polling pauses; logins back off	`503 upstream_unavailable` + `Retry-After`	after 60 s
Shamel slow	Requests queue; background work is throttled first	`503 upstream_busy` / `504` + warning	after 5 min
All Connectors offline	2 s grace, then fast-fail; Connectors reconnect with backoff (losing one fails over in ~2 s)	`503 upstream_unavailable` + `Retry-After`	after 2 min
PostgreSQL down	Reads continue on the last compiled config (up to 6 h), then fail closed; writes fail closed	reads OK; writes `503`	after 60 s

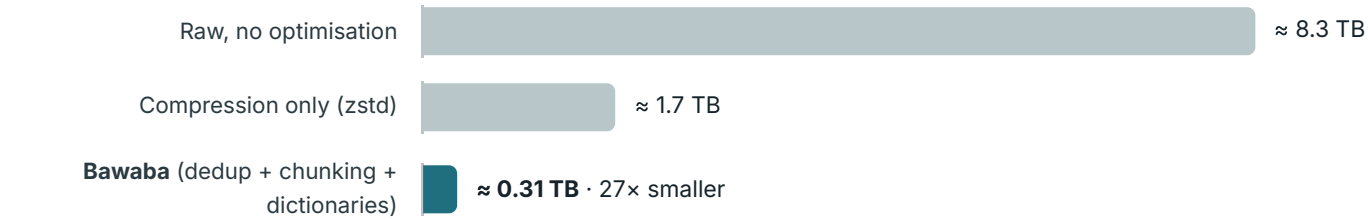
13 AUDIT LOG

Record everything, store very little

You asked for a **full** audit: every request and response, encrypted. Recording that naively would grow by terabytes. Bawaba stores each unique piece of content once, splits large lists into chunks so that a changed list only stores its changed parts, then compresses and encrypts.

Storage at “medium” load (1,000 requests per minute), default retention

Estimated steady-state size of the audit log, all stores combined.



Per day, bodies: 85.8 GB raw → 12.45 GB compressed → **1.41 GB** stored (≈ 61× smaller). Polling the same 1 MB list 1,680 times a day costs 1.7 GB raw but only about 4–8 MB stored.

How a response is stored

1. **Strip framing** — bytes kept exactly as sent
2. **Chunk** — record-aligned for big lists
3. **Keyed hash** — HMAC id, no content guessing
4. **Seen before?** — store a reference only
5. **zstd** — trained dictionaries
6. **AES-256-GCM** — into immutable packs

Kept	For	Where
Request metadata (who, what, when, IP, status, timing)	2 years	90 days in PostgreSQL, then compact cold segments
Write bodies (invoices, vouchers…)	2 years	blob store, encrypted
Read bodies	90 days	blob store, encrypted
Sandbox bodies	14 days	blob store, encrypted

All retention periods are adjustable. Size caps raise alerts at 70 / 85 / 95 %; at the hard cap only the oldest read (and rejected/sandbox) bodies are trimmed, recorded as a gap, never writes or metadata.

Never silently dropped. If storage falls behind, records spool to an encrypted local disk buffer; if that fills, Bawaba by default fails closed rather than skipping the audit.

Readable only by you. Searching and decrypting happen only in the control plane; every view or export is audited too.

61× smaller daily body storage than raw

27× smaller total audit at steady state

2 yrs of full write history (invoices, vouchers) by default

0 records skipped silently: Bawaba fails closed instead

14 WEBHOOKS & GUIDANCE

Changes come to you, and the API coaches its callers

Shamel can't push changes, so Bawaba checks for them every **5 minutes**, only for events someone has subscribed to, and delivers signed webhooks. Each response also tells the calling tool, often written by an AI, how to behave better.

Webhook events

`customer.created` `customer.updated` `customer.removed` `customer.balance_changed` `item.created` `item.updated` `item.removed` `stock.changed` `voucher.created` `voucher.updated` `voucher.removed` `order.status_changed` `approval.resolved`

- Signed with HMAC-SHA256 (`Bawaba-Signature`), with a timestamp to block replays
- Retries with backoff for about 38 hours, then a dead-letter queue you can replay
- Filtered by the subscribing token's current permissions and hidden fields
- Push targets must be hosts you approved; optional pull mode for tools without a public endpoint

How changes are detected

Customers, items	Shamel's `lastupdatedate` with overlap windows + record hashes; daily full reconciliation
Stock, balances	Diff of the full snapshot
Vouchers	Tiered date-range scans
Order status	Status checks for tracked orders

Only one node polls each stream, so adding nodes never multiplies the load on Shamel. Not everything is seen within 5 min: back-dated vouchers within 1 h, older voucher edits within 24 h, deletions within 48 h; edits to vouchers older than 35 days are not detected.

"Inform the AI" instead of only blocking it

```
# a tool polling too fast
HTTP/1.1 200 OK
# tightest of 3 buckets:
upstream units
RateLimit-Limit: 600
RateLimit-Remaining: 420
RateLimit-Reset: 18
Bawaba-Warning:
  polling_too_frequent;
  observed=8;recommended=60,
  use_webhooks;
  events="customer.updated"
```

Warning	Sent when a tool…
`polling_too_frequent`	repeats the same query faster than recommended
`use_webhooks`	polls for changes that a webhook event already covers
`use_updated_since`	re-downloads full lists instead of asking for changes
`use_cursor_snapshot`	re-fetches page 1 instead of following the cursor
`large_unfiltered_request`	asks for an unfiltered list of over 5,000 visible records
`duplicate_request`	fires the same call 3+ times within 2 seconds
`retry_too_fast`	retries before `Retry-After` has passed

Docs written for AI

`llms.txt` , an AI integration guide with rules of thumb, and recommended polling intervals in the OpenAPI spec.

You get told too

A misbehaving tool gets a dashboard flag first, then an `abusive_tool` e-mail if it keeps going.

Safety net stays

Generous per-token limits and the global Shamel cap still apply, as a last resort.

15 RUNNING BAWABA

Dashboard for you, CLI for you and your AI

The bilingual dashboard (Arabic / English, full RTL) runs **only on the server**, and you reach it through an SSH tunnel. The `bawaba` CLI does everything the dashboard does, prints JSON for scripts, and has a restricted remote mode that AI agents use with their agent token.

LOCAL OWNER MODE

On the server · all commands

Root-only Unix socket (`sudo bawaba …`) or `bawaba login` with password + TOTP. Approvals, firewall rules, live tokens, keys, audit decryption. None of these exist in remote mode.

REMOTE AGENT MODE

Anywhere · agent token over HTTPS

Only agent-safe commands: read docs and the OpenAPI spec, check its own token and usage, request access, make API calls within its own scopes, manage its own webhooks (if granted).

```
# AI agent, on its own machine
$ bawaba request-token \
    --scopes customers:read,stock:read \
    --env test --reason "stock dashboard" \
    --wait
✓ request areq_19 filed · pending owner

# owner, on the server
$ sudo bawaba access-request list
  areq_19  dash-builder  customers:read
         stock:read    test
$ sudo bawaba access-request approve areq_19
✓ approved · secret can be collected once
```

```
# owner, on the server
$ sudo bawaba token create --kind tool \
    --name sales-dashboard --env live \
    --preset read_only --data-scope store=1,2 \
    --hide-fields items.purchase_price \
    --ip-allow 203.0.113.0/28 --json
{"data":
{"id":"tok_01JBX4…","token":"bwb_live_…"},"meta":
{}}

$ sudo bawaba firewall explain 198.51.100.7
  banned · auto: auth_failure · 24h

$ sudo bawaba audit search \
    --token tok_01JBX4… --since 1h
```

Dashboard pages

Overview
traffic, errors, latency, Shamel health, pending items

Tokens & presets
create, scope, hide fields, IP allowlists, revoke

Access requests
AI requests vs. the agent's own rights; approve or narrow

Write approvals
pending writes shown as real documents; sandbox writes on their own page

Upstreams & Connectors
enroll, confirm, suspend, kill

Firewall
allow/deny rules, bans, "explain this IP"

Audit explorer
search, decision trace, decrypt, diff

Usage analytics
per token and endpoint, warnings, behaviour scores

Webhooks
subscriptions, target-host approvals, poll streams, dead letters

Alerts
rules, thresholds, recipients, silences

System
nodes, keys, backups, restore drills

Built simply
server-rendered HTML + htmx, no npm, strict CSP

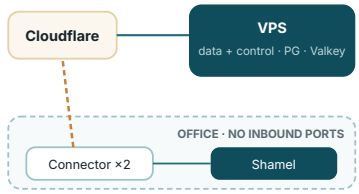
16 DEPLOYMENT

Three ways to run it, one codebase

Hosting is still to be decided, so Bawaba ships as Docker Compose bundles and standalone binaries. The gateway runs on Linux; the Connector also runs as a Windows service, since Shamel likely runs on a Windows server.

RECOMMENDED

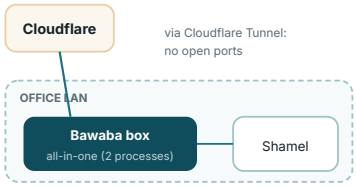
A · Cloud VPS + Connector



- Two layers of hiding: Cloudflare hides the VPS, the tunnel hides Shamel
- Shamel passwords stay at the office, with the Connector
- No office network changes; easy to back up

SIMPLE

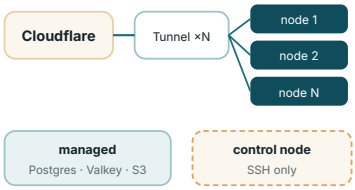
B · One box in the office



- Everything on one Linux machine next to Shamel
- Exposed only through a Cloudflare Tunnel
- Depends on office power and internet; needs an off-site backup

SCALE

C · Scaled-out cluster



- Identical stateless data nodes; scale by CPU
- Separate small control node, reached only by SSH
- Rolling upgrades with draining

Artifact	Linux	Windows	Docker image
`bawaba-gateway`	yes (primary)	—	distroless, non-root, read-only filesystem
`bawaba-connector`	yes (systemd)	**yes, as a Windows service**	available
`bawaba` **CLI**	yes	yes (agent commands)	inside the gateway image
`bawaba-mock`	yes	yes	available

Backups

Encrypted PostgreSQL archives (7 daily, 4 weekly, 12 monthly) and incremental blob sync. Monthly restore drills are tracked.

Master key backup

Only `master.key` needs one (`data.key` is re-issued from it): paper export or 2-of-3 shares, kept apart from data backups.

Upgrades

Expand/contract migrations; cluster nodes drain one at a time with zero downtime (a plain single-box restart costs seconds).

Which one?

Situation	Topology
Shamel is on an office LAN and the Connector can be installed	A
Everything should stay in the office and a spare Linux box or VM is available	B
Sustained heavy load, or high availability needed for the gateway itself	C
The Connector can't be installed, but Shamel is reachable over a VPN or an allowlisted IP	A or C, direct-URL mode

17 DELIVERY PLAN

Reads first, then writes, then scale

As you asked, everything that **reads** data comes first. AI agents can start building against the sandbox partway through M1, your first tools go live in a restricted pilot, and writes and webhooks follow in parallel.

M0 — Foundations

Repo, CI with the leak suite, config, two key files, Postgres/Valkey/blob wiring, mock Shamel with Arabic data, OpenAPI skeleton, Docker bundles

Exit: `GET /v1/currencies` through all 17 pipeline stages

M1 — Read API

Every read endpoint, tokens & permissions, firewall, origin hiding, snapshots, Arabic search, full audit, Connector, CLI, dashboard, docs, alerts

Exit: leak suite clean, pentest round 1 with no open critical/high

M2 — Writes

Write endpoints for all 13 voucher types, idempotency, approval queue, batches, legacy writes, AI access requests

Exit: 0 duplicates in 10,000 fault-injected writes

M3 — Expansions

Webhooks engine and API, usage analytics UI, remaining alerts, Grafana bundles

Exit: no polling without subscribers; no lost events

M4 — Hardening

Load tests against the targets, chaos tests for every failure mode, cluster topology, pentest round 2, restore drills

Exit: all performance targets met on the reference node

Milestone 1 ships in three steps

STEP 1

Sandbox reads (α)

AI agents start building your read tools with `bwb_test_` tokens against fake data.

STEP 2

Live read pilot (β)

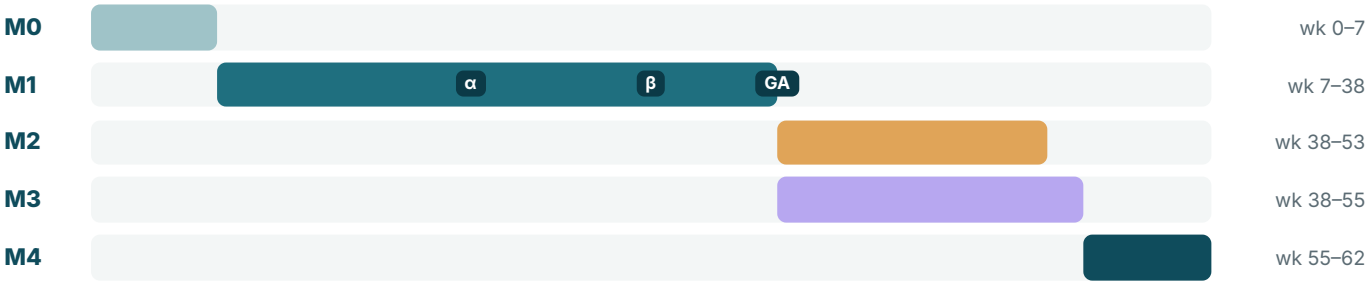
Your first tools on real data, with live tokens locked to your own IP addresses, for at least 14 days.

STEP 3

All reads live (GA)

After the pilot and the first pentest. The API contract is frozen at version 1.0.0.

Rough calendar (3 senior engineers)



About 154 engineer-weeks (184 with 20 % contingency, ±25 %): roughly 14 months with 3 engineers or 11 months with 4. The estimate assumes a human team and **no** AI speed-up; AI agents can shorten the coding, but not the reviews, security testing or live verification. It should be re-baselined after M0.

ISCO answers gate some steps. Connectivity, sessions and limits (set A) gate the live pilot; read semantics (B) gate M1 GA; write semantics (C) gate live writes per voucher type; change detection (D) gates webhooks.

Decisions still needed from you: hosting, domain, Cloudflare account and SMTP before the pilot; whether any existing integration needs the legacy mode; team size.

18 QUESTIONS FOR ISCO

55 questions, ready to send

The manual leaves real gaps, such as how long a Shamel session lasts, what `GetSalesmen` returns, and whether HTTPS is supported. The design works around each one safely, but ISCO's answers will settle them. `08-isco-questions.md` has a polite cover letter and every question in **Arabic and English**, each with a "why we ask" line.

The 10 priority questions

Id	Topic	الموضوع
ISCO-01	Session token lifetime and concurrent sessions per user	مدة صلاحية رمز الجلسة والجلسات المتزامنة للمستخدم
ISCO-05	HTTPS / TLS support	دعم HTTPS وTLS
ISCO-18	Error format, stable error codes, expired-token signal	صيغة استجابات الأخطاء ورموز الأخطاء الثابتة
ISCO-20	Rate and concurrency limits, recommended parallel calls	حدود معدل الطلبات والطلبات المتزامنة
ISCO-21	Pagination, truncation and large responses	تقسيم النتائج إلى صفحات والاستجابات الكبيرة
ISCO-22	Meaning of `lastupdatedate`	دلالة المعامل lastupdatedate
ISCO-26	Response of `/Files/GetSalesmen`	استجابة GetSalesmen
ISCO-43	Duplicate prevention and outcome of failed writes	منع التكرار ونتيجة عمليات الإدخال الفاشلة
ISCO-50	Shamel version, API versioning, change notices	إصدار الشامل وإدارة إصدارات الواجهة البرمجية
ISCO-53	Licensing: internal layer now, commercial use later	الترخيص: طبقة التكامل الداخلية حالياً والاستخدام التجاري مستقبلاً

All 16 topic groups

4 Authentication & sessions	8 Transport security & IIS hosting	6 Request & response conventions	2 Errors
2 Load, limits & large responses	3 Incremental sync & change detection	1 Salesman scoping	4 Master data
4 Customers	3 Items, stock & catalog	3 Reports	3 Vouchers: reading
7 Writes	1 Enumerations	3 Versions, test environment & roadmap	1 Licensing & commercial use

Before you send it: fill in your company name, confirm ISCO's official Arabic name, and decide whether to mention possible commercial use later (ISCO-53 c). A few "why we ask" lines mention internal limits you may want to trim.

When answers arrive: record them in the document. Each question links back to the design sections it affects, so the matching defaults can be updated.

19 DECISIONS

What you decided

The design follows your answers from the requirements interview. These are recorded in `docs/design/00-decisions.md`, which every other document must stay consistent with.

Audience (v1)	Your own company; selling it comes later
Token users	Tools and dashboards you build, mostly with AI
Shamel servers	One now, more later
Language	Go: chosen for speed and load
Load	Medium now, built for heavy; hardware is the only limit
Hosting	Decide later; Docker + binaries
Reaching Shamel	Connector tunnel (preferred) + direct URL
Edge	Cloudflare in front
Master key	Root-only key files on the server (master + data)
API style	Clean `/v1` + optional legacy mode
Data freshness	Always live; 60-second paging snapshots
Overload	Inform the AI first; global Shamel cap as safety net
First milestone	Everything that reads data

Shamel login	Configurable per token
Permissions	Scopes, voucher types, data scope, field hiding, presets
Token extras	IP allowlist, usage analytics; no expiry by default
Browser tokens	Supported, off by default
AI agents	Request → you approve on the server
Sandbox	Yes, with fake data
Write safety	Approval queue + undo-friendly audit
Admin	Localhost dashboard (ar/en) + CLI with JSON
Alerts	E-mail
Audit	Everything, full, encrypted, optimized
Webhooks	Change checks every 5 minutes
Billing / licensing	Not in v1

The design documents

00 · Decisions
The record of your answers

01 · Architecture
Components, pipeline, Connector, scaling, failures

02 · Security & firewall
Threat model, origin hiding, crypto, AI containment

03 · Public API
Every endpoint, errors, paging, writes, webhooks API

04 · Tokens & permissions
Scopes, data scope, field hiding, approvals

05 · Operations
Audit, webhooks engine, alerts, dashboard, CLI, backups

07 · Milestones
Delivery plan and acceptance criteria

08 · ISCO questions
English + Arabic list to send to ISCO

Upstream reference
The Shamel Pro API manual, transcribed (there is no doc 06)